LAPORAN PRAKTIKUM

ALGORITMA PEMROGRAMAN 2

MODUL 5 – REKRUSIF

NAMA : Muhammad Ervan Febriano

NIM : 109082500079

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. Struktur dasar program go (Package main, func main)
2. Tipe data dan variable dalam go
3. Operator dan instruksi dasar (Assignment, input/output)
4. Perulangan menggunakan for

B. Guided

1. [bodymassindex.go]

```go
package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)
    baris(n)
}
func baris(bilangan int) {
    if bilangan == 1 {
        fmt.Println(1)
    } else {
        fmt.Println(bilangan)
        baris(bilangan - 1)
    }
}

```

Output :

```
PS C:\Users\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Mu
\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Erva
d 1.go"
1
1
PS C:\Users\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Mu
\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Erva
d 1.go"
5
5
4
3
2
1
```

2. [targetTabungan.go]

```go
package main
import "fmt"

func main() {
    var n int
    fmt.Scan(&n)
    fmt.Println(penjumlahan(n))
}
func penjumlahan(n int) int {
    if n == 1 {
        return 1
    } else {
        return n + penjumlahan(n-1)
    }
}
```

Output :

```
PS C:\Users\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhar
\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervan_F
d2.go"
4
10
PS C:\Users\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhar
\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervan_F
d2.go"
5
15
```

3. [kategoriTiket.go]

```go
package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)
    fmt.Println(pangkat(n))
}
func pangkat(n int) int {
    if n == 0 {
        return 1
```

```
13)     } else {
14)         return 2 * pangkat(n-1)
15)     }
16)}
4.
```

Output :

```
PS C:\Users\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_
\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervan_Febri
d3.go"
5
32
PS C:\Users\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_
\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervan_Febri
d3.go"
3
8
```

C. Unguided

1. Soal Unguided 1

Deret fibonacci adalah sebuah deret dengan nilai suku ke-0 dan ke-1 adalah 0 dan 1, dan nilai suku ke-n selanjutnya adalah hasil penjumlahan dua suku sebelumnya. Secara umum dapat diformulasikan ⍰⍰ = ⍰⍰−1 + ⍰⍰−2 . Berikut ini adalah contoh nilai deret fibonacci hingga suku ke-10. Buatlah program yang mengimplementasikan fungsi rekursif pada deret fibonacci tersebut.

Jawaban :

```
1)      package main

2)      import "fmt"

3)      func fibonacci(n int) int {
4)      if n == 0 {
a.      return 0
5)      } else if n == 1 {
a.      return 1
6)      }
7)      return fibonacci(n-1) + fibonacci(n-2)
8)      }

9)      func main() {
10)     var n int
11)     fmt.Print("Masukkan jumlah suku: ")
12)     fmt.Scan(&n)
```

```
    fmt.Println("Deret Fibonacci:")
    for i := 0; i < n; i++ {
    fmt.Print(fibonacci(i), " ")
    }
    }
```

Output :

```
PS C:\Users\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Erva
\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervan_Febriano\
nguided1.go"
Masukkan jumlah suku: 5
Deret Fibonacci:
0 1 1 2 3
PS C:\Users\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Erva
\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervan_Febriano\
nguided1.go"
Masukkan jumlah suku: 4
Deret Fibonacci:
0 1 1 2
```

Penjelasan :

[Program di atas menggunakan konsep rekursif, yaitu fungsi yang memanggil dirinya sendiri untuk menyelesaikan masalah yang lebih kecil. Fungsi fibonacci(n) bekerja berdasarkan definisi deret Fibonacci, di mana setiap suku merupakan penjumlahan dari dua suku sebelumnya. Untuk menghentikan rekursi agar tidak berjalan tanpa batas, digunakan kondisi dasar (base case), yaitu saat n == 0 menghasilkan 0 dan n == 1 menghasilkan 1. Jika nilai n lebih besar dari 1, maka fungsi akan memanggil dirinya sendiri dengan parameter n-1 dan n-2, lalu menjumlahkan hasilnya. Pada fungsi main, program meminta input jumlah suku yang ingin ditampilkan, kemudian menggunakan perulangan untuk mencetak deret Fibonacci dari suku ke-0 hingga suku ke-(n-1). Pendekatan ini mudah dipahami karena mengikuti definisi matematika secara langsung, namun memiliki kelemahan dari segi efisiensi karena banyak perhitungan yang diulang]

2. Soal Unguided 2

Buatlah program yang mengimplementasikan rekursif untuk menampilkan faktor bilangan dari suatu N, atau bilangan yang apa saja yang habis membagi N.

Masukan terdiri dari sebuah bilangan bulat positif N.
Keluaran terdiri dari barisan bilangan yang menjadi faktor dari N (terurut dari 1 hingga N ya).

Jawaban :

```go
package main

import "fmt"

func faktor(n int, i int) {
    if i > n {
        return
    }

    if n%i == 0 {
        fmt.Print(i, " ")
    }

    faktor(n, i+1)
}

func main() {
    var n int
    fmt.Print("Masukkan bilangan: ")
    fmt.Scan(&n)

    fmt.Println("Faktor dari", n, "adalah:")
    faktor(n, 1)
}
```

Output :

```
PS C:\Users\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervar
\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervan_Febriano\1
empCodeRunnerFile.go"
Masukkan bilangan: 5
Faktor dari 5 adalah:
1 5
PS C:\Users\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervar
\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervan_Febriano\1
empCodeRunnerFile.go"
Masukkan bilangan: 3
Faktor dari 3 adalah:
1 3
```

Penjelasan :

Program ini menggunakan konsep rekursi untuk mencari semua faktor dari sebuah bilangan N. Fungsi faktor(n, i) bekerja dengan cara mengecek setiap bilangan mulai dari 1 hingga Nmenggunakan parameter i sebagai pencacah. Jika i lebih besar dari n, maka proses rekursi dihentikan (base case). Pada setiap pemanggilan fungsi, program akan memeriksa apakah n % i == 0, yang berarti i adalah faktor dari n, lalu mencetaknya. Setelah itu, fungsi memanggil dirinya sendiri dengan nilai i+1 untuk melanjutkan pengecekan ke bilangan berikutnya. Di dalam main, pengguna diminta memasukkan nilai N, kemudian fungsi rekursif dipanggil mulai dari i = 1. Dengan cara ini, semua faktor dari Nakan ditampilkan secara berurutan dari yang terkecil hingga terbesar.

3. Soal Unguided 3

Buatlah program yang mengimplementasikan rekursif untuk menampilkan barisan bilangan ganjil.

Masukan terdiri dari sebuah bilangan bulat positif N.
Keluaran terdiri dari barisan bilangan ganjil dari 1 hingga N.

Jawaban :

```go
package main

import "fmt"

func ganjil(n int, i int) {
    if i > n {
        return
    }

    if i%2 != 0 {
        fmt.Print(i, " ")
    }

    ganjil(n, i+1)
}

func main() {
    var n int
    fmt.Print("Masukkan bilangan: ")
    fmt.Scan(&n)
```

```
    fmt.Println("Bilangan ganjil dari 1 sampai", n, "adalah:")
    ganjil(n, 1)
}
```

Output :

```
PS C:\Users\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Erva
\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervan_Febriano\
nguided3.go"
Masukkan bilangan: 6
Bilangan ganjil dari 1 sampai 6 adalah:
1 3 5
PS C:\Users\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Erva
\USER\Documents\ENGGINER\Alpro\PRAKTIKUM\109082500079_Muhammad_Ervan_Febriano\
nguided3.go"
Masukkan bilangan: 10
Bilangan ganjil dari 1 sampai 10 adalah:
1 3 5 7 9
```

Penjelasan :

Program ini menggunakan rekursi untuk menampilkan bilangan ganjil dari 1 hingga N. Fungsi ganjil(n, i) berfungsi sebagai proses utama yang memeriksa setiap bilangan dari 1 sampai Nmenggunakan parameter i. Jika i lebih besar dari n, maka rekursi dihentikan (base case). Pada setiap langkah, program mengecek apakah i adalah bilangan ganjil dengan kondisi i % 2 != 0. Jika benar, maka nilai i akan ditampilkan. Setelah itu, fungsi memanggil dirinya sendiri dengan nilai i+1 untuk melanjutkan ke bilangan berikutnya. Di dalam main, pengguna memasukkan nilai N, lalu fungsi dipanggil mulai dari i = 1. Dengan cara ini, semua bilangan ganjil dalam rentang tersebut akan ditampilkan secara berurutan.

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa rekursif merupakan metode yang efektif untuk menyelesaikan permasalahan yang memiliki pola berulang, seperti deret Fibonacci, pencarian faktor bilangan, dan penentuan bilangan ganjil. Setiap program rekursif harus memiliki base-case sebagai kondisi berhenti dan recursive-case sebagai proses pemanggilan ulang fungsi. Tanpa base-case, program akan berjalan terus menerus tanpa henti. Implementasi rekursif juga memperlihatkan bahwa suatu permasalahan dapat diselesaikan dengan cara yang lebih sederhana dan terstruktur, meskipun dari segi efisiensi terkadang lebih lambat dibandingkan metode iteratif. Oleh karena itu, pemahaman konsep dasar rekursif sangat penting agar dapat digunakan secara tepat dalam pemrograman.